is also a lot of information that the user is expected to input which needs to be encrypted and secured such that the database cannot be hacked into and compromised. In this chapter we will study the methodologies that are employed by the framework to keep it secure. We will learn the syntax that exists in the framework with examples wherever possible to bring out the various security functions in a Laravel script. Let us begin.

Authentication

Authentication is an extremely important aspect of creating a web application. Laravel authentication works on two main pillars:

1. Guards: define user authentication on every request.
2. Providers: define how users are retrieved from their persistent storage.

This is what any basic authentication and registration form looks like

Laravel comes with several pre-built authentication controllers, which are located in the App*Http\Controllers\Auth* namespace. The 'RegisterController' handles new user registration, the 'LoginController' handles authentication, the 'ForgotPasswordController' handles email links for resetting passwords, and the 'ResetPasswordController' contains the logic to reset passwords. Each of these controllers uses a trait to include their necessary methods. For most of the web applications, these traits do not need any kind of modification. There is a quick way to scaffold the routes and views that would be needed for authentication. The syntax for the same is:

```
Php artisan make : auth
```

With this command, all the views will be created and stored in the 'resources/views/auth' directory.

Now that the views are created and stored in the directory, the process of registry and authentication for the application is ready to be done. A new database needn't be created after this is done as the webpage has already been initialized in the browser by the preset traits that are stored in the controllers that we used.